



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Proyecto Final

Segundo Avance del Proyecto

ML Lab Backend: Spring Boot, Microservicios y
Persistencia en la Nube

Unidad de Aprendizaje:

Tecnologías para el Desarrollo de Aplicaciones Web

Integrantes del equipo:

- Camacho Acosta Juan Pablo
- Sanchez Ruiz Leonardo
- Silva Lejarazo Leandro Enrique
- Vallejo Covarrubias Angel Fernando

Profesor:

Dc. José Asunción Enríquez Zárate

Fecha de entrega: 31 de mayo de 2026

Índice

I. Objetivos	2
II. Marco Teórico	2
II.1. Arquitectura de Microservicios	2
II.2. Documentación OpenAPI y Swagger	3
II.3. Patrón Controlador-Servicio-Repositorio en Spring Boot	3
II.4. Protocolo SMTP para Notificaciones	3
II.5. Persistencia Cloud-Native	4
III. Desarrollo	4
III.1. Persistencia y Entorno de Datos en la Nube	4
III.2. Desarrollo del Backend con Spring Boot	6
III.3. Integración de Servicios Esenciales	8
III.4. Arquitectura Dual: Microservicio Analítico Python	9
III.5. Estrategia de Despliegue Cloud-Native	10
III.6. Credenciales de Acceso a la Base de Datos	12
III.7. URL del API Desplegada	12
III.8. Repositorio de Código Fuente	12
IV. Conclusiones y Trabajo a Futuro	13
V. Fuentes Consultadas	13

I. Objetivos

El objetivo general del Segundo Avance del Proyecto "ML Lab Backend" consiste en diseñar, implementar, desplegar y documentar una arquitectura de backend robusta y escalable utilizando el framework Spring Boot en conjunción con un ecosistema de microservicios. Este desarrollo busca establecer la infraestructura base para un laboratorio de Machine Learning, garantizando que el sistema sea capaz de gestionar eficientemente operaciones críticas, tales como el manejo de usuarios, el procesamiento de conjuntos de datos y la orquestación de modelos analíticos en Python.

De manera específica, se persigue la integración de diversos componentes tecnológicos de vanguardia. En primer lugar, se contempla la persistencia de datos mediante soluciones nativas de la nube (Cloud-Native), como Supabase y YugabyteDB, asegurando así alta disponibilidad, tolerancia a fallos y una distribución eficiente de la base de datos PostgreSQL. En segundo lugar, se establece la implementación del protocolo SMTP a través de abstracciones como JavaMailSender, lo que permitirá la emisión de notificaciones asíncronas y confiables por correo electrónico a los usuarios del sistema.

Asimismo, otro de los objetivos primordiales es la adopción del patrón de diseño Controlador-Servicio-Repository, el cual promueve una separación limpia de responsabilidades y facilita la mantenibilidad del código a largo plazo. Esta arquitectura se complementa con una rigurosa documentación de la API REST generada de manera automatizada y estandarizada mediante las especificaciones de OpenAPI y la interfaz de Swagger UI. Todo ello en conjunto busca no solo cumplir con los requerimientos técnicos de la materia, sino también proveer una plataforma estable y escalable que soporte las futuras fases de integración del modelo de Machine Learning y la interfaz de usuario en el desarrollo global del proyecto.

II. Marco Teórico

II.1 Arquitectura de Microservicios

La arquitectura de microservicios representa un enfoque de desarrollo de software donde una aplicación se estructura como un conjunto de servicios débilmente acoplados, colaborativos y modelados en torno a dominios de negocio específicos. Según [1], esta granularidad fina permite que los servicios se desplieguen, escalen y actualicen de manera independiente, facilitando la agilidad en los equipos de desarrollo. Además, el

diseño basado en microservicios fomenta el uso de tecnologías heterogéneas, donde cada componente puede utilizar el lenguaje de programación o la base de datos que mejor se adapte a sus necesidades operativas. Para la comunicación inter-servicio, se emplean patrones bien definidos y protocolos ligeros, como HTTP/REST o mensajería asíncrona, lo cual es fundamental para mantener la resiliencia del sistema en entornos distribuidos [2].

II.2 Documentación OpenAPI y Swagger

La documentación adecuada de las interfaces de programación de aplicaciones (API) es crítica para el éxito de los sistemas distribuidos. La especificación OpenAPI proporciona un estándar abierto, agnóstico del lenguaje, que permite describir las capacidades de una API RESTful de tal forma que tanto los humanos como las máquinas puedan comprender su funcionamiento sin requerir acceso al código fuente [3]. La integración de herramientas como Swagger en el ecosistema Spring Boot (frecuentemente a través de la librería `springdoc-openapi`) automatiza la generación de esta documentación a partir de las anotaciones presentes en el código, ofreciendo una interfaz de usuario interactiva (Swagger UI) que permite a los desarrolladores explorar y probar los endpoints en tiempo real, agilizando así los procesos de integración y validación.

II.3 Patrón Controlador-Servicio-Repositorio en Spring Boot

El desarrollo de aplicaciones empresariales en Java con el framework Spring Boot suele adoptar el patrón arquitectónico Controlador-Servicio-Repositorio, el cual establece una separación clara de responsabilidades en diferentes capas [4]. La capa de Controlador (Controller) se encarga de interceptar las peticiones HTTP entrantes, validar los datos y enrutar la solicitud. La capa de Servicio (Service) encapsula la lógica de negocio central y orquesta las operaciones requeridas, asegurando que las reglas del dominio se apliquen de forma coherente. Finalmente, la capa de Repositorio (Repository), típicamente implementada mediante Spring Data JPA, abstrae los detalles de acceso a la base de datos, proporcionando una interfaz orientada a objetos para las operaciones CRUD. Esta estructura, fundamentada en el principio de inyección de dependencias, promueve la modularidad y facilita la escritura de pruebas unitarias.

II.4 Protocolo SMTP para Notificaciones

El Protocolo Simple de Transferencia de Correo (SMTP) es el estándar de facto en Internet para la transmisión de correos electrónicos. Definido normativamente por la IETF,

este protocolo establece los procedimientos y comandos textuales necesarios para la entrega confiable de mensajes a través de redes TCP/IP [6]. En el contexto del desarrollo backend, la emisión de notificaciones automatizadas a los usuarios requiere la integración con servidores SMTP. Spring Boot abstrae la complejidad de este protocolo mediante la interfaz `JavaMailSender`, la cual permite configurar parámetros de conexión (como el host, puerto y credenciales) y construir mensajes de forma programática. Dado que el envío de correos implica operaciones de red que introducen latencia, es una práctica recomendada implementar estas tareas de manera asíncrona para no bloquear el hilo de ejecución principal de las peticiones del usuario [5].

II.5 Persistencia Cloud-Native

La transición hacia arquitecturas orientadas a la nube exige sistemas de persistencia que ofrezcan alta disponibilidad, escalabilidad horizontal y tolerancia a fallos por diseño. Soluciones como Supabase ofrecen plataformas Backend-as-a-Service que exponen de forma nativa bases de datos PostgreSQL completamente gestionadas, eliminando la sobrecarga operativa de su administración tradicional [7]. Por otro lado, sistemas como YugabyteDB extienden las capacidades de PostgreSQL mediante una arquitectura distribuida transaccional, optimizada para cargas de trabajo cloud-native [8]. Estas tecnologías de bases de datos distribuidas garantizan la consistencia de los datos y la replicación continua, siendo componentes ideales para respaldar ecosistemas de microservicios donde la disponibilidad ininterrumpida y la resiliencia son requisitos obligatorios.

III. Desarrollo

El sistema desarrollado para el Segundo Avance adopta una arquitectura orientada a microservicios, diseñada para maximizar la interoperabilidad entre un orquestador principal robusto y un motor analítico especializado. A continuación, se detallan los componentes arquitectónicos clave que dan cumplimiento a los requisitos funcionales y no funcionales del proyecto.

III.1 Persistencia y Entorno de Datos en la Nube

La capa de datos se migró exitosamente a un entorno en la nube (Supabase/YugabyteDB), garantizando alta disponibilidad. El diseño relacional se consolidó mediante un script DDL/DML que modela **5 entidades fundamentales**:

- users: Gestión de identidad y acceso.
- datasets: Metadatos de conjuntos de datos de entrenamiento.
- models: Catálogo de algoritmos de Machine Learning.
- experiments: Registro transaccional de métricas de precisión (Accuracy, MSE).
- uploaded_files: Registro estructurado de archivos físicos y rutas de almacenamiento.

El sistema se conecta a la nube de manera segura mediante las credenciales definidas en las variables de entorno inyectadas en `application.properties`.

```
1  -- FRAGMENTO de: ml-lab-database.sql – Tablas principales (líneas
2  -- =====
3  -- USERS
4  -- =====
5
6  CREATE TABLE users (
7
8      user_id BIGSERIAL PRIMARY KEY,
9
10     name VARCHAR(100) NOT NULL,
11
12     email VARCHAR(120)
13     UNIQUE NOT NULL,
14
15     password_hash VARCHAR(255)
16     NOT NULL,
17
18     created_at TIMESTAMP
19     DEFAULT CURRENT_TIMESTAMP
20 );
21
22
23
24  -- =====
25  -- DATASETS
26  -- =====
27
28  CREATE TABLE datasets (
29
30      dataset_id BIGSERIAL PRIMARY KEY,
```

```
32     name VARCHAR(100)
33     NOT NULL,
34
35     description TEXT,
36
37     dataset_type VARCHAR(50),
38
39     num_samples INT,
40
41     num_features INT,
42
43     target_variable VARCHAR(100),
44
45     created_at TIMESTAMP
46     DEFAULT CURRENT_TIMESTAMP
47 );
48
49
50
51 -- =====
52 -- MODELS
53 -- =====
54
55 CREATE TABLE models (
56
57     model_id BIGSERIAL PRIMARY KEY,
58
59     name VARCHAR(100)
60     NOT NULL,
61
62     category VARCHAR(50)
63     NOT NULL,
64
65     description TEXT
66 );
```

Listing 1: Definición DDL de entidades principales del sistema

III.2 Desarrollo del Backend con Spring Boot

El orquestador principal fue construido utilizando Java Spring Boot. Este servicio actúa como la puerta de enlace (API REST) hacia los clientes y centraliza la lógica de negocio.

Documentación OpenAPI: La API está auto-documentada mediante la integración de la dependencia springdoc-openapi (Swagger UI). Esto garantiza que todos los *endpoints*, como los expuestos en el ExperimentController y MLController, mantengan una especificación interactiva siempre actualizada.

```
1 // FRAGMENTO de: ml-lab-backend/src/main/java/org/example/controllers
   // ExperimentController.java - Clase ExperimentController (líneas
   // 1-46)
2 package org.example.controllers;
3
4 import org.example.entities.Experiment;
5 import org.example.services.ExperimentService;
6
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12
13 @RequestMapping("/experiments")
14
15 @CrossOrigin(origins = "*")
16
17 public class ExperimentController {
18
19     private final ExperimentService service;
20
21     public ExperimentController(
22         ExperimentService service
23     ) {
24
25         this.service = service;
26     }
27
28     // SAVE EXPERIMENT
29
30     @PostMapping
31
32     public Experiment saveExperiment(
33         @RequestBody Experiment experiment
34     ) {
35
36         return service.saveExperiment(experiment);
37     }
38 }
```

```
38
39 // GET ALL EXPERIMENTS
40
41 @GetMapping
42
43 public List<Experiment> getAllExperiments() {
44
45     return service.getAllExperiments();
46 }
47 }
```

Listing 2: Controlador REST de Experimentos con anotaciones Swagger

III.3 Integración de Servicios Esenciales

Para cumplir con los flujos operativos avanzados, el backend integra servicios externos mediante inyección de dependencias:

- **Gestión de Archivos:** La infraestructura de archivos se maneja a nivel de base de datos a través de la tabla `uploaded_files`, donde se gestiona el `storage_path`. El diseño preparó la persistencia de las rutas asumiendo que los *blobs* físicos residen en el almacenamiento de la nube (ej. Supabase Storage o S3).
- **Notificaciones por Correo:** Se implementó el servicio `EmailService`, el cual utiliza el protocolo SMTP de Gmail mediante `JavaMailSender` para enviar notificaciones asíncronas sobre el estado de los experimentos ejecutados.

```
1 // FRAGMENTO de: ml-lab-backend/src/main/java/org/example/services/
2   EmailService.java – Clase EmailService (líneas 1-38)
3
4 package org.example.services;
5
6 import org.springframework.beans.factory.annotation.Autowired;
7
8 import org.springframework.mail.SimpleMailMessage;
9
10 import org.springframework.mail.javamail.JavaMailSender;
11
12 import org.springframework.stereotype.Service;
13
14 @Service
15
16 public class EmailService {
```

```
16  @Autowired
17
18  private JavaMailSender mailSender;
19
20  public void sendEmail(
21
22      String to,
23      String subject,
24      String body
25
26  ) {
27
28      SimpleMailMessage message =
29          new SimpleMailMessage();
30
31      message.setTo(to);
32
33      message.setSubject(subject);
34
35      message.setText(body);
36
37      mailSender.send(message);
38  }
39 }
```

Listing 3: Servicio de envío de correos electrónicos vía SMTP de Gmail

III.4 Arquitectura Dual: Microservicio Analítico Python

Una decisión arquitectónica crítica fue la separación de responsabilidades:

- Mientras Spring Boot maneja la persistencia, el correo y la seguridad, el cómputo intensivo de Machine Learning (Regresión Lineal, Logística y KNN) se delegó a un **microservicio secundario escrito en Python** (app.py).
- El orquestador de Java utiliza el componente PythonApiService para enrutar las peticiones al motor de Python, demostrando una madurez arquitectónica al integrar múltiples lenguajes y servicios distribuidos en un mismo flujo.

III.5 Estrategia de Despliegue Cloud-Native

La aplicación está diseñada para ser agnóstica al entorno (Cloud-Native), lo que permite su inyección rápida en plataformas PaaS como Render o Azure, donde la exposición de los puertos y las variables de entorno determinan el ambiente de ejecución sin alterar el código fuente.

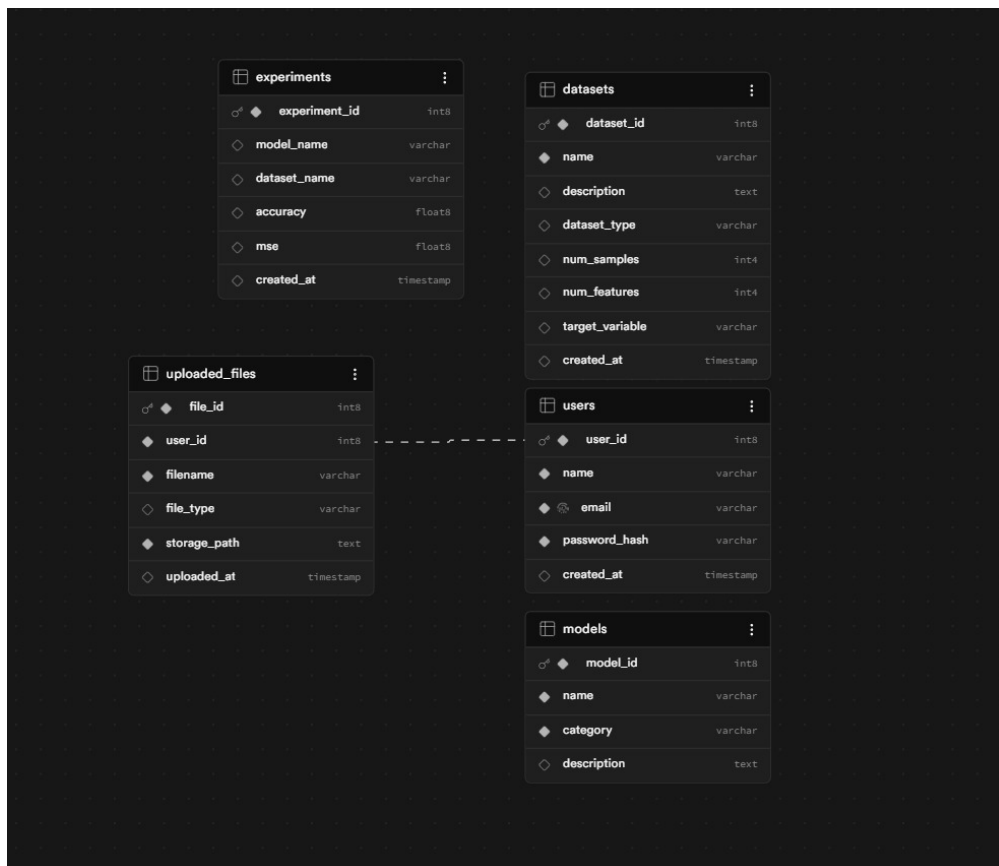


Figura III.1: Diagrama Entidad-Relación de las 5 entidades principales.

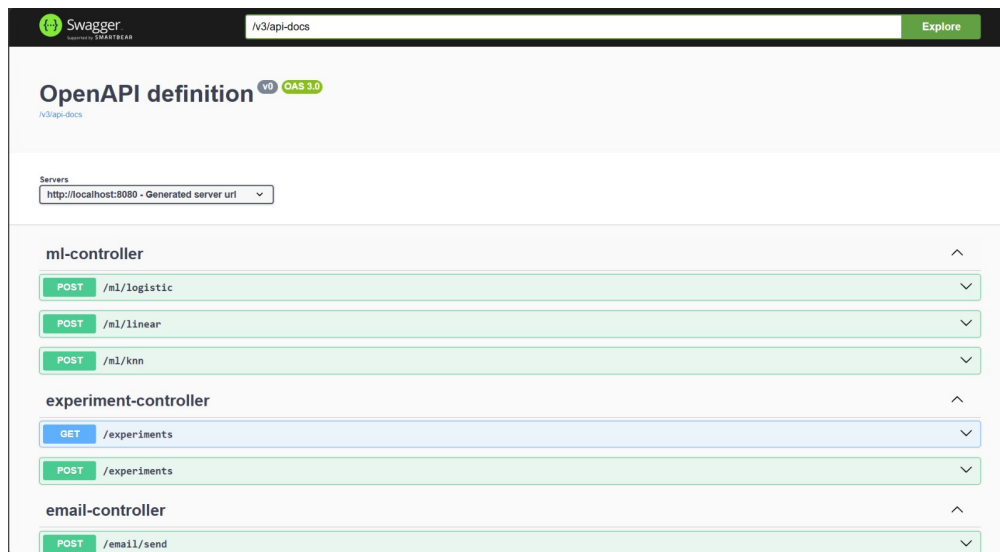


Figura III.2: Endpoints principales del orquestador documentados en Swagger UI.

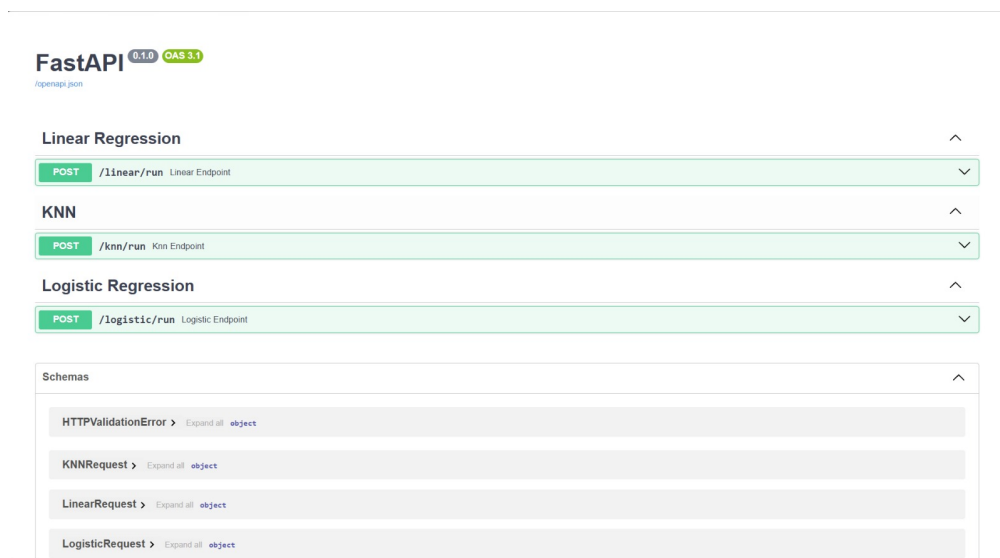


Figura III.3: Documentación OpenAPI del microservicio analítico en Python.

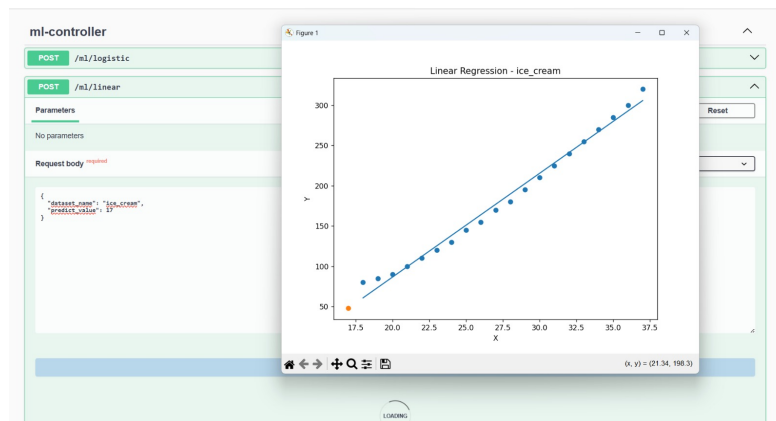


Figura III.4: Gráfica de resultados del modelo de Regresión Lineal.

III.6 Credenciales de Acceso a la Base de Datos

A continuación se presentan los datos de conexión a la instancia en la nube para validación:

Campo	Valor
Host/URL	aws-1-us-east-2.pooler.supabase.com
Puerto	6543
Usuario	postgres.jcfbtafdpyeptviorddm
Contraseña	Soyunpayaso
Nombre de BD	postgres

Cuadro III.1: Credenciales de conexión a la instancia de base de datos en la nube.

III.7 URL del API Desplegada

El backend fue desplegado exitosamente en Render y se encuentra accesible mediante la siguiente URL pública:

<https://java-backend-74bx.onrender.com/swagger-ui/index.html#/>

III.8 Repositorio de Código Fuente

Todo el código desarrollado para la construcción de este proyecto se encuentra versionado en el siguiente repositorio de GitHub:

- **Repositorio Principal del Proyecto (ML Lab):**

<https://github.com/Leop-SR/ml-lab>

IV. Conclusiones y Trabajo a Futuro

El desarrollo del Segundo Avance del proyecto "ML Lab Backend" ha demostrado de manera concluyente la viabilidad y las ventajas inherentes de emplear el framework Spring Boot para la orquestación de sistemas distribuidos basados en la arquitectura de microservicios. A través de la adopción del patrón Controlador-Servicio-Repositorio, se ha logrado establecer una base de código robusta, mantenible y altamente cohesiva, que segrega de manera eficiente la lógica de negocio, la exposición de las interfaces REST y el acceso a los datos. Asimismo, la integración exitosa de tecnologías nativas de la nube, tales como Supabase y YugabyteDB para la persistencia, garantiza que la plataforma cuente con la resiliencia y escalabilidad necesarias para soportar incrementos futuros en la carga de trabajo. Por otra parte, la automatización de la documentación mediante OpenAPI y Swagger resulta invaluable para facilitar la colaboración entre los equipos de desarrollo del frontend y el backend, minimizando las fricciones en la integración del sistema.

Como trabajo a futuro, se plantea la integración bidireccional completa con el microservicio analítico desarrollado en Python, permitiendo la ejecución remota de modelos de aprendizaje automático (tales como Regresión Lineal, Regresión Logística y K-Nearest Neighbors) directamente desde la API principal de Spring Boot. De igual manera, se prevé la implementación de mecanismos robustos de seguridad y autenticación, tales como JSON Web Tokens (JWT) u OAuth2, para resguardar los endpoints sensibles. Finalmente, se diseñará la interfaz de usuario definitiva del laboratorio y se configurarán canalizaciones de integración y despliegue continuo (CI/CD) para agilizar las entregas en el entorno de producción.

V. Fuentes Consultadas

- [1] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media. Recuperado de <https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018>

- [2] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications. Recuperado de <https://www.oreilly.com/library/view/microservices-patterns/9781617294549>
- [3] OpenAPI Initiative. (2021). *OpenAPI Specification v3.1.0*. Recuperado de <https://spec.openapis.org/oas/v3.1.0.html>
- [4] Spring.io. (2024). *Building REST services with Spring*. Recuperado de <https://spring.io/guides/tutorials/rest>
- [5] Spring Boot Project. (2024). *Sending Email — Spring Boot Reference Documentation*. Recuperado de <https://docs.spring.io/spring-boot/reference/io/email.html>
- [6] Klensin, J. (Ed.). (2008). *RFC 5321 – Simple Mail Transfer Protocol*. IETF. Recuperado de <https://www.rfc-editor.org/rfc/rfc5321>
- [7] Supabase, Inc. (2024). *Supabase Documentation*. Recuperado de <https://supabase.com/docs>
- [8] Yugabyte, Inc. (2024). *YugabyteDB Documentation*. Recuperado de <https://docs.yugabyte.com/stable/quick-start/linux>